

DARTree: Speculative Diffusion Decoding with Autoregressive Draft Trees

Tianyi Li, Yaxin Luo, Xinyi Shang, Zhiqiang Shen

VILA Lab, MBZUAI

{Tianyi.Li, Yaxin.Luo, Xinyi.Shang, Zhiqiang.Shen}@mbzuai.ac.ae

Code: <https://github.com/VILA-Lab/DARTree>

Abstract

Speculative decoding losslessly accelerates autoregressive language models by verifying multiple draft tokens in parallel. Diffusion-based drafters further reduce proposal latency by predicting an entire token block in parallel, but their position-wise distributions are marginal rather than conditioned on tokens selected along each draft path. Existing recurrent correction incorporates causal information along a single draft chain, whereas diffusion-based tree construction broadens candidate coverage without carrying this correction along individual branches. We introduce DARTree, a training-free speculative decoding method that extends a pretrained AR correction head from chains to trees. DARTree first constructs a fixed-width candidate tree by expanding and scoring all nodes at each depth in a single batch, and then only applies best-first pruning to select the verification tree, decoupling AR-head inference from sequential heap operations. Across seven math, code, and chat benchmarks, DARTree achieves the highest average acceptance length and speedup in all four model-temperature configurations, accepting up to 12.97 tokens per verification round, 98.6% more than DFlash and 27.9% more than Domino in the same setting, and reaching up to $9.73\times$ lossless speedup over locally measured autoregressive decoding.

Introduction

While modern autoregressive (AR) large language models (LLMs) have achieved remarkable success in understanding, reasoning, and generating text and code (Achiam et al. 2023; Guo et al. 2025), their inherently sequential nature limits inference speed, resulting in noticeable latency for users. At each decoding step, the model generates only one token per sequence while requiring access to nearly all model parameters, making inference predominantly I/O-bound and wasting the parallel computation capabilities of modern hardware.

To accelerate inference and improve hardware utilization, Speculative Decoding (Leviathan, Kalman, and Matias 2023; Chen et al. 2023) uses a lightweight drafter to propose several future tokens and verifies them in parallel with the target model, preserving the target model’s output distribution. However, autoregressive drafters still produce proposals sequentially, so their latency grows with draft length. Diffusion-based block drafters remove this rollout by predicting an entire block in one forward pass, simultaneously lowering proposal latency and enabling higher-capacity drafters that

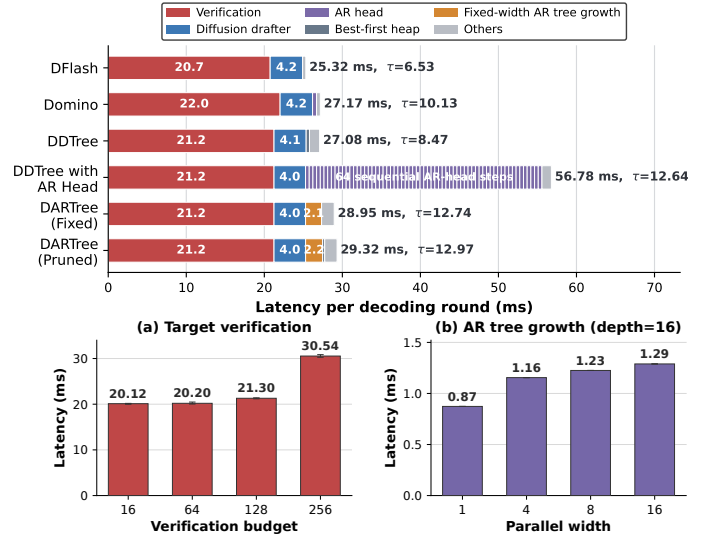


Figure 1: Latency breakdown and effects of verification budget and parallel width on GSM8K (Qwen3-4B, $T = 0$). DARTree avoids sequential AR-head expansion and scales efficiently with budget and parallel width.

improve proposal quality and acceptance, as demonstrated by DFlash (Chen, Liang, and Liu 2026).

This parallelism comes at the cost of causal conditioning. Each draft position is predicted from the verified prefix but not from the realized tokens at earlier positions in the same block, creating a mismatch with the target model’s conditioned distributions. Lightweight causal correction heads mitigate this mismatch by propagating selected tokens along the draft chain, partially restoring the missing intra-block dependencies without running the parallel backbone (Huang et al. 2026; Cheng et al. 2026).

Draft-tree verification provides a separate way to improve speculative decoding by exposing multiple candidate continuations to the target model. A prefix tree can be scored in one forward pass using tree attention, improving candidate coverage when an early draft position is uncertain. DDTree (Ringel and Romano 2026) constructs such a tree from the position-wise distributions of a block-parallel diffusion drafter; under a fixed node budget, its best-first search

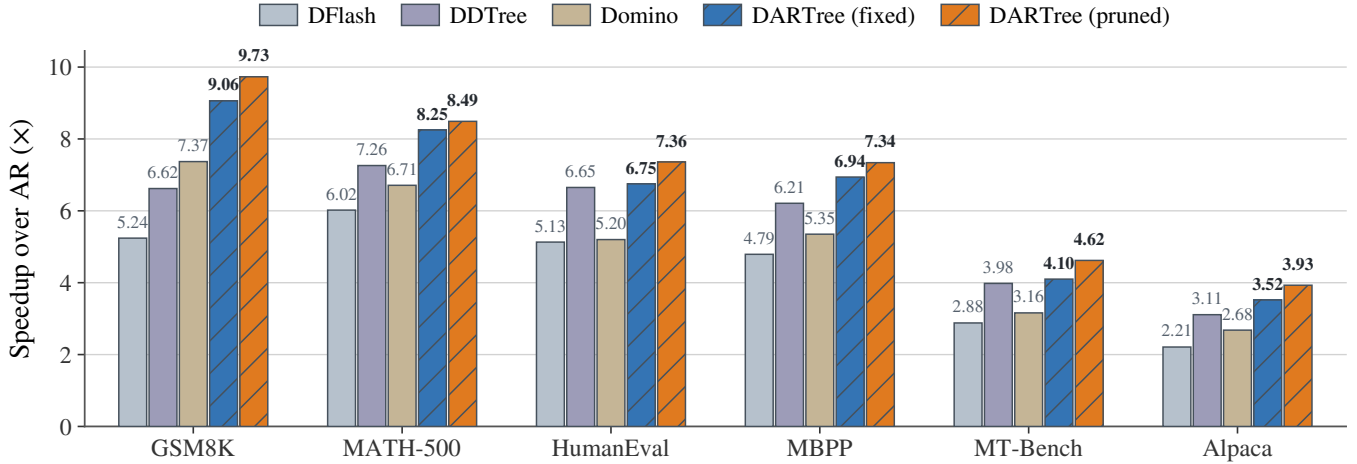


Figure 2: Speculative decoding speedup over standard autoregressive decoding across six benchmarks. DARTree with fixed-width construction and subsequent pruning consistently outperforms prior diffusion-based speculative decoding methods.

selects the highest-mass prefixes to maximize the surrogate expected acceptance length with a max heap. Draft trees and causal correction therefore address complementary limitations: the former broadens candidate coverage, while the latter improves causal dependency across draft positions.

Combining causal correction with this heap-based construction, however, creates a new efficiency bottleneck. Because the corrected scores of a node’s children depend on that node’s path-specific state, each heap pop must be followed by correction-head inference and subsequent heap pushes before the next node can be expanded. This interleaving forces correction-head inference to proceed one node at a time, making tree construction a substantial source of latency, as shown in Figure 1. The key challenge is to retain path-dependent scoring while batching multiple branches.

We introduce **DARTree**, a training-free speculative decoding method that extends a pretrained causally corrected block-parallel drafter from a single chain to a speculative tree. DARTree expands a fixed number of nodes at each depth and evaluates their corrected distributions in one batch, avoiding the node-wise correction interleaved with heap operations. Since wider layer expansion adds little latency, DARTree first constructs a wider supertree and then applies best-first pruning later to obtain a compact verification tree, decoupling correction-head inference from sequential heap operations.

Across seven math, code, and chat benchmarks with Qwen3-4B and Qwen3-8B at temperatures 0 and 1, DARTree achieves the highest overall average acceptance length and speedup in all four model-temperature configurations. Figure 2 summarizes the decoding speedups across representative benchmarks. Our contributions are threefold:

- We identify the sequential bottleneck caused by coupling path-conditioned causal correction with node-wise best-first tree construction.
- We introduce a training-free, depth-wise batched tree-construction method that applies causal correction across multiple branches in parallel, achieving lossless speedup.
- We propose a deferred and simplified best-first pruning

strategy for selecting the final verification tree.

Related Work

Speculative Decoding. Speculative decoding accelerates autoregressive generation by drafting multiple future tokens and verifying them in parallel through the target model with rejection sampling (Leviathan, Kalman, and Matias 2023; Xia et al. 2023; Chen et al. 2023). Standard multi-token prediction (MTP) can serve as a lightweight drafting mechanism, using auxiliary prediction heads to propose tokens at multiple future positions in a single forward pass (Gloeckle et al. 2024). Medusa (Cai et al. 2024) augments the target model with multiple prediction heads and organizes their candidates into a tree for parallel verification. The EAGLE family instead employs a learned drafter conditioned on internal features of the target model. EAGLE (Li et al. 2024b) autoregressively predicts the target model’s hidden states and maps them to candidate-token distributions using the target model’s output head; EAGLE-2 (Li et al. 2024a) dynamically allocates draft-tree nodes under a fixed node budget using context-dependent confidence scores; and EAGLE-3 (Li et al. 2025b) replaces feature prediction with direct token prediction using fused multi-layer target-model features.

Diffusion Language Models. Diffusion language models (DLMs) generate sequences through iterative denoising and update multiple tokens in parallel, offering an alternative to left-to-right autoregressive generation (Austin et al. 2021a; Nie et al. 2025; Ye et al. 2025; Li et al. 2025a; Myrzakhan et al. 2026; Gong et al. 2022). Recent masked DLMs such as LLaDA (Nie et al. 2025) demonstrate that this paradigm can scale to large language models, while block diffusion generates autoregressively across blocks and denoises tokens within each block in parallel, thereby recovering compatibility with KV caching (Arriola et al. 2025). However, the parallelism of DLMs introduces a fundamental speed-quality trade-off: accepting more tokens per denoising step improves throughput but often degrades generation quality, whereas conservative decoding requires many refinement steps (Li

et al. 2025a; Wu et al. 2025). Moreover, repeated bidirectional computation makes standard KV caching difficult and leads to unfavorable scaling on long sequences. These limitations motivate using DLMs as efficient draft generators under autoregressive verification, rather than as standalone replacements for autoregressive models.

Diffusion-Based Parallel Drafting. Diffusion-based drafting exploits sequence-level parallelism to reduce proposal latency (Christopher et al. 2025). DFlash (Chen, Liang, and Liu 2026) introduces a lightweight block drafter that generates an entire proposal block in a single forward pass, achieving strong drafting quality and acceleration. Plain one-pass diffusion-based drafters ignore intra-block causal dependencies; Domino (Huang et al. 2026) and DSpark (Cheng et al. 2026) partially recover them with lightweight causal correction heads. A complementary line organizes parallel draft predictions into trees. DDTree (Ringel and Romano 2026) derives a globally optimal prefix tree under a fixed node budget with respect to its diffusion-marginal surrogate, using best-first heap expansion. In its primary Domino instantiation, DARTree integrates these directions by extending full-prefix AR correction from a single draft chain to multiple tree branches. It expands and scores candidate branches depth-wise in parallel and, after completing the candidate supertree, applies a single global top- B selection to form the verification tree, avoiding sequential heap operations during correction.

Preliminaries

Speculative Sampling

Let p and q denote the target and draft models, respectively, and let $x_{1:t}$ be the tokens committed before a speculative decoding round. The drafter proposes γ tokens $y_{1:\gamma}$, where $y_{<i} = (y_1, \dots, y_{i-1})$ and $c_i = (x_{1:t}, y_{<i})$ is the context at draft position i . When $T = 0$, verification accepts consecutive draft tokens that match the target model’s greedy predictions. At the first mismatch, it emits the target prediction and terminates the current round.

When $T > 0$, let p_T and q_T be the temperature-scaled target and draft distributions. For $y_i \sim q_T(\cdot | c_i)$, standard speculative sampling accepts y_i with probability

$$a_i = \min\left(1, \frac{p_T(y_i | c_i)}{q_T(y_i | c_i)}\right). \quad (1)$$

If accepted, y_i is committed and verification continues to position $i + 1$. At the first rejection, only one replacement token is sampled from

$$r_T(v | c_i) = \frac{[p_T(v | c_i) - q_T(v | c_i)]_+}{\sum_{w \in \mathcal{V}} [p_T(w | c_i) - q_T(w | c_i)]_+}, \quad (2)$$

where v denotes a possible replacement token, w indexes the vocabulary $\mathcal{V}$ for normalization, and $[z]_+ = \max(z, 0)$. Every round commits the accepted draft prefix followed by one additional output token as a bonus. At the first rejection, this token is sampled from r_T , the remaining proposals are discarded, and the round ends; if all γ proposals are accepted, it is sampled directly from p_T . In either case, the committed

sequence follows the target distribution (Leviathan, Kalman, and Matias 2023; Chen et al. 2023).

This rejection-sampling procedure is not used by all implementations. DFlash (Chen, Liang, and Liu 2026) instead performs simple exact target-sample matching: it accepts consecutive draft tokens while they match the corresponding target samples, emits the target sample at the first mismatch, and terminates the round. This verification procedure preserves the target distribution without explicitly computing draft–target probability ratios or residual corrections.

Parallel Drafting and Causal Correction

Block-parallel drafters predict the distributions of all γ future positions in one forward pass. Given a verified prefix $x_{1:t}$, these predictions induce the factorized distribution

$$q_{\text{par}}(y_{1:\gamma} | x_{1:t}) = \prod_{i=1}^{\gamma} q_i(y_i | x_{1:t}). \quad (3)$$

By eliminating sequential draft rollout, block-parallel drafting not only reduces proposal latency but also accommodates a higher-capacity drafter, thereby improving proposal quality and acceptance, as demonstrated by DFlash (Chen, Liang, and Liu 2026). However, each q_i conditions only on $x_{1:t}$, whereas the target distribution $p(y_i | x_{1:t}, y_{<i})$ also depends on the realized earlier draft tokens. This missing intra-block dependency is the causal mismatch of parallel drafting.

To mitigate this mismatch, causal correction introduces dependencies among parallel predictions. Markov correction models the one-step transition from y_{i-1} to y_i (Cheng et al. 2026), which can also be approximated in parallel (Reinboldt, Berdoz, and Wattenhofer 2026). RNN correction instead propagates a recurrent state r_i that summarizes the realized prefix $y_{<i}$ (Huang et al. 2026; Cheng et al. 2026).

Tree Verification and DDTree

A draft tree is a prefix tree that represents multiple candidate continuations. For verification, its nodes are flattened, and a tree-attention mask allows each node to attend only to the verified context and the tokens on its root-to-node path. The target model can therefore score all branches in one forward pass, after which the accepted root-to-leaf prefix is selected according to the target model.

DDTree (Ringel and Romano 2026) constructs a tree of at most B nodes by maximizing a surrogate expected acceptance length under the factorized block-diffusion distribution q_{par} defined above. DDTree optimizes

$$\max_{\mathcal{T}: |\mathcal{T}| \leq B} \mathbb{E}_{Y_{1:\gamma} \sim q_{\text{par}}(\cdot | x_{1:t})} [\alpha_{\mathcal{T}}(Y_{1:\gamma})]. \quad (4)$$

For a concrete continuation $y_{1:\gamma}$, its acceptance length is $\alpha_{\mathcal{T}}(y_{1:\gamma}) = \max\{d : y_{1:d} \in \mathcal{T}\}$, and $\alpha_{\mathcal{T}}(Y_{1:\gamma})$ is the random length of the longest prefix of $Y_{1:\gamma}$ contained in the tree. For a prefix $u = (u_1, \dots, u_d)$, its mass is $q_{\text{par}}(u | x_{1:t}) = \prod_{i=1}^d q_i(u_i | x_{1:t})$. The expectation above then decomposes into $\sum_{u \in \mathcal{T}} q_{\text{par}}(u | x_{1:t})$. Since an ancestor has no smaller mass than its descendants, the top- B prefixes with the highest probability mass form the optimal tree under this surrogate.

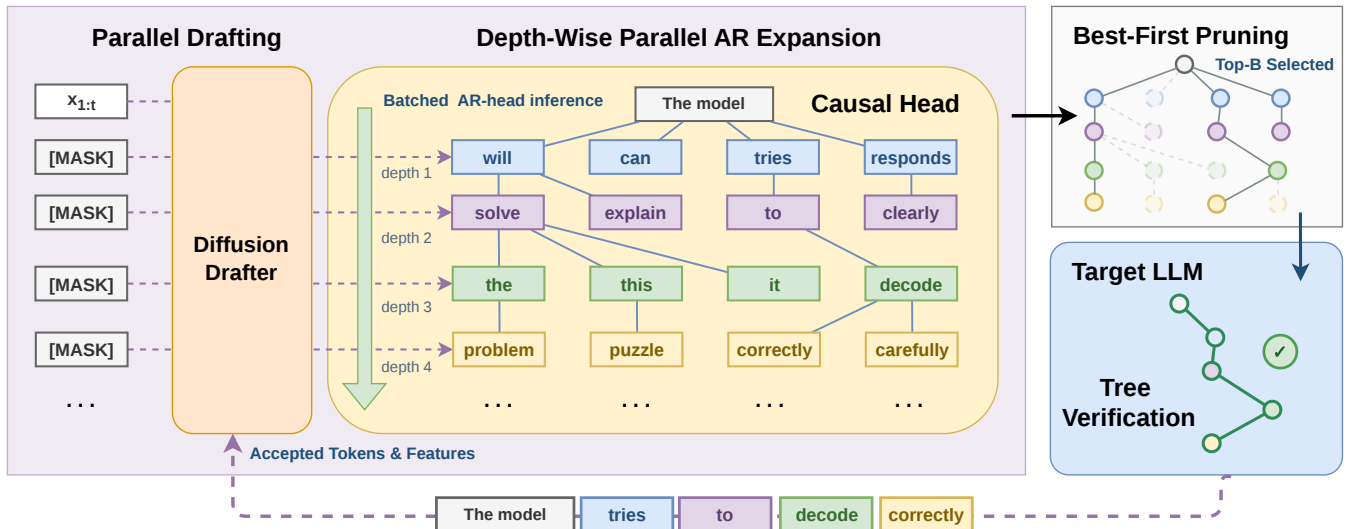


Figure 3: **Overview of DARTree.** Block-parallel drafting is followed by depth-wise batched AR expansion, deferred best-first pruning, and a single standard target-model tree-verification pass. Nodes sharing the same color are generated in the same depth-wise AR-head batch; rounded boxes indicate parameterized modules, while faded nodes are discarded by top- B pruning.

DDTree enumerates them with best-first search using a max-heap: each pop selects the highest-mass prefix and pushes its next sibling and first (highest-probability) child, yielding $O(B \log B)$ complexity for constructing the tree.

Methodology

DARTree is a training-free speculative decoding method that extends a pretrained causally corrected block-parallel drafter from a single chain to a speculative tree, without assuming a particular correction architecture. The central challenge is that branch scores become path-dependent after causal correction, making exact best-first construction inherently sequential. DARTree instead expands a fixed number of nodes in parallel at each depth, constructs a wider candidate supertree, and then prunes it into a compact tree for target-model verification. DARTree modifies only candidate-tree construction; the target-model tree-verification procedure is inherited unchanged from prior work. Figure 3 illustrates this workflow.

Decoupling Causal Correction from Best-First Search

Causal correction makes tree construction path-dependent: the children of a prefix cannot be scored until the correction head has processed the tokens along that prefix. A natural exact approach is to combine these corrected scores with the DDTree-style best-first construction (Ringel and Romano 2026), maintaining a max-heap over candidate prefixes. At each step, the heap pops the highest-scoring prefix, the correction head scores its children under the corresponding token history, and these children are pushed back into the heap before the next prefix can be expanded. This interleaving preserves global best-first order under the corrected scores, but prevents correction from being batched across nodes.

DARTree relaxes the node-wise search order while preserving path-wise causal correction. It evaluates all branches at the same depth together, uses their corrected scores to retain the nodes for the next depth, and carries the selected correction states forward. Search and correction thus remain coupled across depths, but no heap decision separates two corrections within the same depth. The approximation lies only in replacing global, node-wise best-first expansion with a fixed-width, depth-wise schedule; every materialized branch is still scored by the pretrained correction head using its realized prefix.

Depth-Wise Parallel Supertree Construction

The block-parallel backbone is executed once to produce shared representations $\{H_d\}_{d=1}^{\gamma}$ for the entire draft block. To limit correction-head overhead, only its top- K token predictions at each position are considered as candidate extensions. Starting from the root, DARTree batches the active branches at each depth and applies the pretrained correction module to their respective prefix states. It then ranks the candidate extensions by their corrected cumulative scores, retains the highest-scoring extensions under a fixed layer width, and carries the associated correction states to the next depth.

Repeating this depth-wise expansion yields a fixed-width candidate supertree. The fixed layer width gives every depth a regular batched workload: dependencies remain sequential across depths, whereas correction, scoring, selection, and state updates within each depth are tensorized. Because the lightweight correction head is evaluated in batch, increasing the layer width incurs little additional drafting latency, as shown in Figure 1. This makes it practical to construct a supertree wider than the final verification tree. We introduce a verification budget B to control the total number of nodes retained in the final tree passed to the target model. DARTree can therefore construct a wider supertree during drafting and

Algorithm 1: DARTree Construction

Require: Shared representations $\{H_d\}_{d=1}^\gamma$ and base logits $\{L_d^{\text{base}}\}_{d=1}^\gamma$ from one block-parallel pass; correction head g ; candidate-vocabulary size K ; layer width W ; node budget B ; depth bonus $\beta \leq 0$

- 1: Initialize the root layer $S_0 \leftarrow \{\emptyset\}$, root correction state $r(\emptyset) \leftarrow r_0$, and root score $s_\beta(\emptyset) \leftarrow 0$
- 2: Initialize the candidate supertree $\mathcal{S} \leftarrow \emptyset$
- 3: **for** $d = 1$ to γ **do**
- 4: Select $\mathcal{C}_d \leftarrow \text{TopK}(L_d^{\text{base}}, K)$
- 5: Correct all prefixes $u_{<d} \in S_{d-1}$ over $\mathcal{C}_d$ in parallel:
- 6: $\tilde{q}_d(\mathcal{C}_d \mid u_{<d}) \leftarrow g_{\mathcal{C}_d}(H_d, r(u_{<d}))$
- 7: Score each child $u_{1:d} = (u_{<d}, u_d)$ with $u_d \in \mathcal{C}_d$ by
- 8: $s_\beta(u_{1:d}) \leftarrow s_\beta(u_{<d}) + \log \tilde{q}_d(u_d \mid u_{<d}) + \beta$
- 9: Retain the global top- W child prefixes as $\mathcal{S}_d$
- 10: Update their states $r(u_{1:d})$ from $r(u_{<d})$ and u_d in parallel, and set $\mathcal{S} \leftarrow \mathcal{S} \cup \mathcal{S}_d$
- 11: **end for**
- 12: Prune the complete supertree: $\mathcal{T} \leftarrow \text{TopB}_{u \in \mathcal{S}} s_\beta(u)$
- 13: **return** $\mathcal{T}$

prune it to B only after all candidate scores are available.

Deferred Best-First Pruning

Once the supertree is complete, every materialized prefix has a corrected score, so DARTree can recover best-first budget allocation without interleaving heap operations with correction. Let $u_{1:d}$ denote a depth- d prefix and $\tilde{q}_i(u_i)$ the corrected probability of its token at depth i , conditioned on the verified context and its preceding branch tokens. We score each prefix by

$$s_\beta(u_{1:d}) = \sum_{i=1}^d \log \tilde{q}_i(u_i) + \beta d, \quad (5)$$

where β is a depth bonus, with a negative value penalizing deeper prefixes.

Lemma 1 (Heap–top- B equivalence). *Let $\mathcal{S}$ be a materialized prefix tree and B the verification budget. If $\beta \leq 0$ and ancestors are preferred under ties, best-first heap selection (Ringel and Romano 2026) and global top- B selection return the same tree:*

$$\text{BestFirst}_B(\mathcal{S}) = \text{TopB}_{u \in \mathcal{S}} s_\beta(u). \quad (6)$$

Extending a prefix adds a log-probability no greater than zero and a non-positive depth bonus, so no child can score above its parent. The globally highest-scoring B nodes are therefore prefix-closed and are exactly those selected by a best-first max-heap sequentially. Hence, after the supertree has been scored, the heap can be replaced by a single top- B operation. The equivalence need not hold for a positive depth bonus, which may allow a descendant to outrank its ancestor. The resulting prefix tree is then verified by the target model in a single forward pass using a tree-attention mask. Algorithm 1 summarizes the complete depth-wise construction and deferred pruning procedure. In the algorithm, each depth- d node is identified by its root-to-node token prefix $u_{1:d}$; $\mathcal{S}_d$ contains only the retained nodes at depth d , whereas $\mathcal{S}$ accumulates the retained nodes across all depths.

Experiments

Experimental Setup

Models and Benchmarks. Following prior work (Chen, Liang, and Liu 2026; Huang et al. 2026), we evaluate DARTree on Qwen3-4B and Qwen3-8B (Yang et al. 2025). We report the average acceptance length τ , defined as the number of output tokens advanced per draft–verify round, including the bonus token, together with speedup over local AR decoding. Let T_{draft} include draft generation and tree construction, and let T_{verify} denote target verification latency. The amortized per-token latency and speedup are

$$L_{\text{spec}} = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau}, \quad \eta = \frac{L_{\text{AR}}}{L_{\text{spec}}}. \quad (7)$$

Consistent with the evaluation protocols used by existing methods, we first compute internal decoding latency per sample and then macro-average across samples. Our benchmarks span three domains: **Math**: GSM8K (Cobbe et al. 2021), MATH-500 (Lightman et al. 2023), and AIME25 (Zhang and Math-AI 2025); **Code**: HumanEval (Chen et al. 2021), MBPP (Austin et al. 2021b); and **Chat**: MT-Bench (Zheng et al. 2023) and Alpaca (Taori et al. 2023).

Baselines and Implementation. We evaluate DARTree with vanilla AR decoding and representative diffusion-based drafters: DFlash (Chen, Liang, and Liu 2026), which predicts a 16-token block in parallel; DDTree (Ringel and Romano 2026), which builds a best-first tree from DFlash’s marginal predictions; and Domino (Huang et al. 2026), which jointly trains a DFlash backbone with an autoregressive correction head. By default, DARTree uses the released Domino correction head without additional training. We evaluate two variants: **DARTree (fixed)** assigns the same number of nodes to each depth and directly uses the resulting tree for verification, whereas **DARTree (pruned)** constructs a wider supertree and subsequently applies deferred pruning to meet the target-verification budget. Both DARTree variants use $K = 64$ candidate tokens per position. Unless otherwise specified, DDTree and both DARTree variants use the same $B = 64$ target-verification budget. DARTree (fixed) distributes this budget uniformly across 16 draft depths, retaining 4 nodes at each depth. For DARTree (pruned), we use a supertree width of 12 and a pruning depth bonus of $\beta = -0.2$. We additionally examine the effects of the layer width and verification budget through hyperparameter sweeps.

Evaluation Details. We evaluate greedy decoding at $T = 0$ and, following DFlash (Chen, Liang, and Liu 2026) and DDTree (Ringel and Romano 2026), sample from the target model at $T = 1$: verification accepts the longest sampled prefix in the draft tree and emits the target sample at the first mismatch. We use batch size one and generate at most 2048 tokens per prompt. All methods and baselines are measured on one NVIDIA RTX 6000 Ada Generation GPU.

Main Results

Table 1 shows that our DARTree (fixed) already outperforms both DDTree and Domino on the overall averages in all four model–temperature configurations. It improves τ by up to

Model	Method	MATH						CODE				CHAT				OVERALL	
		GSM8K		MATH-500		AIME25		HumanEval		MBPP		MT-Bench		Alpaca		Avg.	
Temperature = 0		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3 4B	DFlash	5.24×	6.53	6.02×	7.81	5.78×	7.21	5.13×	6.59	4.79×	6.08	2.88×	4.43	2.21×	3.11	4.58×	5.97
	DDTree	6.62×	8.47	7.26×	9.71	6.73×	8.99	6.65×	8.74	6.21×	8.13	3.98×	6.01	3.11×	4.39	5.79×	7.78
	Domino	7.37×	10.14	6.71×	9.05	5.72×	7.38	5.20×	7.04	5.35×	7.10	3.16×	5.22	2.68×	3.87	5.17×	7.11
	DARTree (fixed)	9.06×	12.74	8.25×	11.64	7.20×	10.05	6.75×	9.45	6.94×	9.57	4.10×	6.84	3.52×	5.23	6.55×	9.36
	DARTree (pruned)	9.73×	12.97	8.49×	12.05	7.45×	10.54	7.36×	9.94	7.34×	10.08	4.62×	7.34	3.93×	5.72	6.99×	9.81
Qwen3 8B	DFlash	4.99×	6.55	5.93×	7.93	5.59×	7.25	5.02×	6.58	4.51×	5.99	2.68×	4.25	2.18×	3.12	4.41×	5.95
	DDTree	5.94×	8.44	6.84×	9.76	6.17×	8.81	6.08×	8.61	5.55×	7.95	3.52×	5.80	2.92×	4.42	5.29×	7.68
	Domino	7.23×	10.00	6.61×	9.36	5.40×	7.48	5.34×	7.34	5.06×	7.05	3.04×	5.18	2.50×	3.80	5.03×	7.17
	DARTree (fixed)	8.47×	12.57	7.89×	11.86	6.91×	10.20	6.66×	9.81	6.33×	9.56	3.82×	6.85	3.25×	5.25	6.19×	9.44
	DARTree (pruned)	8.64×	12.93	8.14×	12.25	7.14×	10.55	6.98×	10.37	6.71×	10.09	4.24×	7.43	3.57×	5.74	6.49×	9.91
Temperature = 1		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3 4B	DFlash	4.60×	5.94	5.14×	6.67	3.81×	4.91	4.79×	6.01	4.35×	5.60	2.59×	4.00	2.15×	3.00	3.92×	5.16
	DDTree	5.96×	7.92	6.49×	8.63	4.92×	6.80	6.19×	8.03	5.84×	7.71	3.60×	5.50	2.95×	4.14	5.14×	6.96
	Domino	6.26×	8.32	5.20×	7.14	3.43×	4.70	4.71×	6.20	4.99×	6.26	2.69×	4.49	2.36×	3.33	4.23×	5.78
	DARTree (fixed)	8.03×	11.17	6.63×	9.60	4.50×	6.56	6.30×	8.65	6.39×	8.85	3.73×	6.20	3.19×	4.72	5.54×	7.96
	DARTree (pruned)	8.26×	11.63	6.92×	10.10	5.01×	7.37	6.76×	9.11	6.88×	9.33	4.09×	6.61	3.61×	5.23	5.93×	8.48
Qwen3 8B	DFlash	4.44×	5.92	4.60×	6.34	3.54×	4.81	4.22×	5.54	3.97×	5.29	2.41×	3.78	2.08×	3.01	3.61×	4.96
	DDTree	5.47×	7.82	5.76×	8.43	4.56×	6.66	5.38×	7.64	5.01×	7.22	3.19×	5.21	2.80×	4.24	4.60×	6.75
	Domino	5.79×	8.14	4.85×	7.10	3.40×	4.78	4.22×	5.87	4.14×	5.84	2.49×	4.28	2.13×	3.20	3.86×	5.60
	DARTree (fixed)	7.31×	11.04	6.07×	9.45	4.43×	6.76	5.45×	8.06	5.45×	8.21	3.29×	5.84	2.96×	4.73	4.99×	7.73
	DARTree (pruned)	7.34×	11.12	6.41×	9.88	4.50×	6.98	5.65×	8.42	5.68×	8.63	3.61×	6.29	3.22×	5.16	5.20×	8.07

Table 1: **DARTree results on Qwen3-4B and Qwen3-8B.** We report decoding speedup over locally measured vanilla autoregressive decoding and average acceptance length (τ , the number of tokens accepted per verification round). The **best** result in each model and temperature block is highlighted in bold.

22.9% over DDTree and 37.9% over Domino, with corresponding speedup gains of up to 17.0% and 30.8%. This demonstrates that extending causal correction from a single chain to multiple depth-wise branches provides substantial gains even without adaptive tree pruning. Furthermore, it is evident that *DARTree (pruned)* achieves the highest overall average τ and speedup in all four configurations, outperforming DDTree and Domino by up to 28.9% and 46.8% in τ , and by up to 22.7% and 40.1% in speedup, respectively. The only exception for individual benchmarks is AIME25 with Qwen3-8B at $T = 1$, where DDTree is marginally faster (4.56 $\times$ versus 4.50 $\times$), while DARTree still attains a higher acceptance length. Compared with the fixed variant, deferred pruning further improves τ by up to 6.5% and speedup by up to 7.1%, showing the benefit of selecting the verification tree from a wider set of corrected branches. The improvements at $T = 1$ further show that DARTree’s gains extend beyond greedy decoding to stochastic sampling. On GSM8K with Qwen3-4B at $T = 0$, DARTree accepts 12.97 tokens per verification round and reaches a 9.73 $\times$ speedup. Its acceptance length is 98.6% higher than DFlash and 27.9% higher than Domino in the same setting.

Ablation Study

We isolate two aspects of DARTree: its depth-wise construction schedule and its compatibility with different causal-

correction heads. The construction ablations use Qwen3-4B at $T = 0$ and the same 64-node target-verification budget.

Tree-construction Strategy. In Table 2, we compare DARTree with two natural alternatives. Sequential Correction w. Heap interleaves correction-head inference with best-first heap updates. It obtains acceptance lengths close to DARTree, confirming the quality of fully conditional node-wise search, but requires roughly 70 ms per round, more than twice DARTree’s latency on all three datasets. Domino-chain + DDTree first applies causal correction along a single draft chain and then uses the resulting chain-conditioned distributions to construct a tree with DDTree. Although efficient, alternative branches are not corrected according to their own token histories. By applying path-specific correction to multiple branches in parallel, DARTree improves τ over Domino-chain + DDTree by up to 16.4%, while avoiding the sequential correction and heap-update overhead of the first alternative. It therefore achieves the highest speedup on all three datasets.

Correction-head Transfer. To evaluate the extensibility of DARTree across different causally corrected diffusion drafters, we additionally apply it to the Markov-head variant of the recently released diffusion drafter DSpark (Cheng et al. 2026). As shown in Table 3, DARTree improves both metrics in all six dataset–temperature settings: acceptance

Method	τ	Round Time (ms)	Speedup
GSM8K			
Seq. Correction w. Heap	12.840	62.703 $\pm$ 0.695	4.38 $\pm$ 0.05 $\times$
Domino-chain + DDTree	11.272	28.285 $\pm$ 0.207	8.48 $\pm$ 0.06 $\times$
DARTree	12.967	28.494 $\pm$ 0.606	9.77$\pm$0.21$\times$
HumanEval			
Seq. Correction w. Heap	9.827	64.267 $\pm$ 0.876	3.25 $\pm$ 0.04 $\times$
Domino-chain + DDTree	8.539	28.643 $\pm$ 0.276	6.31 $\pm$ 0.06 $\times$
DARTree	9.938	28.805 $\pm$ 0.409	7.36$\pm$0.11$\times$
MT-Bench			
Seq. Correction w. Heap	7.369	65.828 $\pm$ 0.702	2.01 $\pm$ 0.02 $\times$
Domino-chain + DDTree	6.520	29.502 $\pm$ 0.180	3.98 $\pm$ 0.03 $\times$
DARTree	7.343	29.098 $\pm$ 0.437	4.56$\pm$0.07$\times$

Table 2: Ablation of tree construction strategies on Qwen3-4B at temperature 0. Round time and speedup are computed per sample and macro-averaged over each benchmark

Method	Temperature = 0		Temperature = 1	
	τ	Speedup	τ	Speedup
GSM8K				
DSpark-Markov	6.607	5.54 $\times$	5.732	4.66 $\times$
+ DARTree (pruned)	7.571	5.61$\times$	7.319	5.26$\times$
HumanEval				
DSpark-Markov	5.728	4.78 $\times$	5.040	3.97 $\times$
+ DARTree (pruned)	7.112	5.37$\times$	6.820	4.94$\times$
MT-Bench				
DSpark-Markov	4.437	3.31 $\times$	3.881	2.71 $\times$
+ DARTree (pruned)	5.825	3.97$\times$	5.455	3.64$\times$

Table 3: Ablation using the released DSpark with Markov correction head on Qwen3-4B.

length increases by 14.6–40.6%, while speedup improves by up to 34.3%. These results support the view that DARTree is a general tree-construction method for causally corrected parallel drafters rather than a Domino-specific modification.

Influence of Hyperparameters. We study the two important hyperparameters of DARTree on Qwen3-4B at $T = 0$, averaging equally over GSM8K, HumanEval, and MT-Bench. Figure 4(a) examines the effect of layer width while fixing $B = 64$. Increasing the width from 4 to 12 noticeably improves acceptance by preserving more branch alternatives, whereas wider layers provide little additional benefit. Figure 4(b) fixes $W = 12$ and varies the target-verification budget. Larger budgets monotonically improve acceptance over most of the tested range, but speedup peaks around $B = 64$ –128 and falls at $B = 192$. We therefore use $B = 64$ in the main experiments as a balanced operating point.

Acceptance Patterns. Figure 5 examines where the acceptance improvement arises. DARTree maintains a visibly higher probability of reaching later positions: on GSM8K, DARTree accepts almost the entire draft block in nearly half of the decoding rounds. DDTree provides stronger accep-

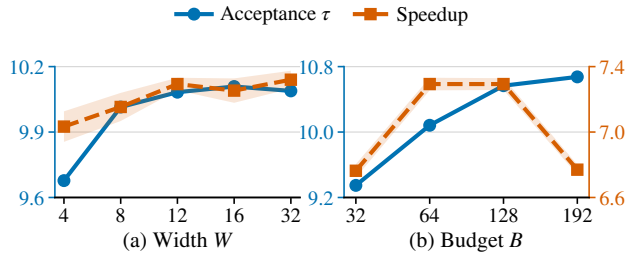


Figure 4: Effect of (a) layer width W with fixed $B = 64$ and (b) verification budget B with fixed $W = 12$, averaged over three benchmarks. Shading indicates propagated standard deviation across four runs.

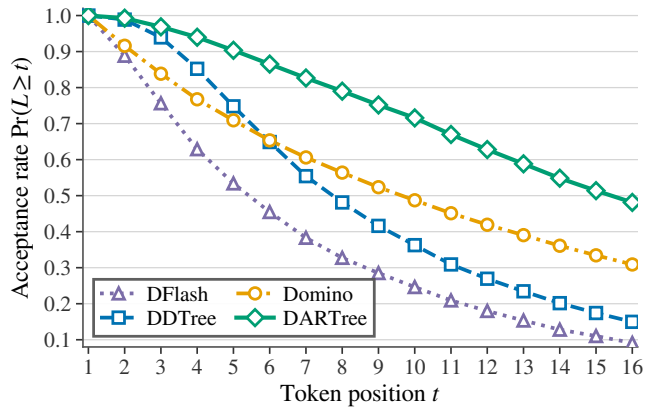


Figure 5: Position-wise acceptance rates on GSM8K. Here, L denotes the acceptance length of a decoding round, and $\Pr(L \geq t)$ is the probability that a decoding round advances at least t output tokens. DARTree is much more likely to reach later positions.

tance at early positions, whereas Domino maintains a higher probability of reaching later positions; DARTree combines both advantages, preserving strong early acceptance while substantially extending the accepted continuation.

Conclusion

We presented DARTree, a training-free speculative decoding method that extends causally corrected block-parallel drafting from a single chain to a tree. DARTree carries path-specific correction across multiple branches through depth-wise batched expansion, and performs deferred best-first pruning with a single global top- B selection after the candidate supertree has been fully scored. This design avoids the node-wise coupling of correction-head inference and sequential heap updates while leaving the standard target-model tree-verification procedure unchanged. Across all evaluated benchmarks, DARTree achieves the highest overall average acceptance length and speedup in every model-temperature configuration, with consistent benefits under both greedy decoding and stochastic sampling.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Arriola, M.; Gokaslan, A.; Chiu, J.; Yang, Z.; Qi, Z.; Han, J.; Sahoo, S.; and Kuleshov, V. 2025. Block diffusion: Interpolating between autoregressive and diffusion language models. In *International Conference on Learning Representations*.
- Austin, J.; Johnson, D. D.; Ho, J.; Tarlow, D.; and van den Berg, R. 2021a. Structured Denoising Diffusion Models in Discrete State-Spaces. *Advances in Neural Information Processing Systems*, 34: 17981–17993.
- Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; et al. 2021b. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.
- Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J. D.; Chen, D.; and Dao, T. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.
- Chen, C.; Borgeaud, S.; Irving, G.; Lespiau, J.-B.; Sifre, L.; and Jumper, J. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.
- Chen, J.; Liang, Y.; and Liu, Z. 2026. DFlash: Block Diffusion for Flash Speculative Decoding. *arXiv preprint arXiv:2602.06036*.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Cheng, X.; Yu, X.; Shao, C.; Li, J.; Xiong, Y.; Qian, Y.; Zhu, J.; Ma, S.; Zhang, X.; Ye, J.; et al. 2026. DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation. *arXiv preprint arXiv:2607.05147*.
- Christopher, J. K.; Bartoldson, B. R.; Ben-Nun, T.; Cardei, M.; Kailkhura, B.; and Fioretto, F. 2025. Speculative diffusion decoding: Accelerating language generation through diffusion. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 12042–12059.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Gloeckle, F.; Idrissi, B. Y.; Rozière, B.; Lopez-Paz, D.; and Synnaeve, G. 2024. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.
- Gong, S.; Li, M.; Feng, J.; Wu, Z.; and Kong, L. 2022. Dif-fuseq: Sequence to sequence text generation with diffusion models. *arXiv preprint arXiv:2210.08933*.
- Guo, D.; Yang, D.; Zhang, H.; Song, J.; Wang, P.; Zhu, Q.; Xu, R.; Zhang, R.; Ma, S.; Bi, X.; et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Huang, J.; Zhang, Y.; Zhang, Q.; Lin, H.; Xu, H.; and Zhang, L. 2026. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*.
- Leviathan, Y.; Kalman, M.; and Matias, Y. 2023. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 19274–19286. PMLR.
- Li, T.; Chen, M.; Guo, B.; and Shen, Z. 2025a. A survey on diffusion language models. *arXiv preprint arXiv:2508.10875*.
- Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024a. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, 7421–7432.
- Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024b. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *International Conference on Machine Learning*, 28935–28948. PMLR.
- Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2025b. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test. *ArXiv*, abs/2503.01840.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050*.
- Myrzakhan, A.; Li, T.; Guo, B.; Tang, S.; and Shen, Z. 2026. Sink-aware pruning for diffusion language models. *arXiv preprint arXiv:2602.17664*.
- Nie, S.; Zhu, F.; You, Z.; Zhang, X.; Ou, J.; Hu, J.; Zhou, J.; Lin, Y.; Wen, J.-R.; and Li, C. 2025. Large language diffusion models. *arXiv preprint arXiv:2502.09992*.
- Rheinboldt, P.; Berdoz, F.; and Wattenhofer, R. 2026. TreeFlash: Parallel AR-Approximation for Faster Speculative Decoding. *arXiv preprint arXiv:2606.03819*.
- Ringel, L.; and Romano, Y. 2026. Accelerating speculative decoding with block diffusion draft trees. *arXiv preprint arXiv:2604.12989*.
- Taori, R.; Gulrajani, I.; Zhang, T.; Dubois, Y.; Li, X.; Guestrin, C.; Liang, P.; and Hashimoto, T. B. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. <https://crfm.stanford.edu/2023/03/13/alpaca.html>, 3(6): 7.
- Wu, C.; Zhang, H.; Xue, S.; Liu, Z.; Diao, S.; Zhu, L.; Luo, P.; Han, S.; and Xie, E. 2025. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*.
- Xia, H.; Ge, T.; Wang, P.; Chen, S.-Q.; Wei, F.; and Sui, Z. 2023. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 3909–3925.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Ye, J.; Xie, Z.; Zheng, L.; Gao, J.; Wu, Z.; Jiang, X.; Li, Z.; and Kong, L. 2025. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*.

Zhang, Y.; and Math-AI, T. 2025. American Invitational Mathematics Examination (AIME) 2025.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E.; et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36: 46595–46623.

Appendix

A. Limitations

Despite its consistent performance across a wide range of evaluations, DARTree has several limitations. First, DARTree relies on a pretrained diffusion drafter equipped with a causal correction head. Consequently, its training-free property does not directly apply to naive diffusion drafters, which require additional training to incorporate such a head. This requirement also restricts our evaluation to compatible models publicly available. Moreover, as a speculative decoding method, DARTree does not reduce the total number of FLOPs; instead, it uses additional computation to reduce inference latency. Similar to other tree-based speculative decoding approaches, DARTree incurs even greater computational overhead than non-tree-based methods because it verifies a large tree of candidate tokens. Therefore, it may not be suitable for all deployment environments. We discuss the scenarios in which DARTree is most effective in the following section.

B. When Should We Use DARTree?

Like other tree-based speculative decoding methods, DARTree is primarily designed for low-concurrency serving, where decoding is highly memory-bandwidth-bound and the computational resources remain underutilized. In this regime, additional computation for drafting and tree verification can be traded for lower decoding latency and higher token throughput. As concurrency increases, batched decoding improves hardware utilization and the workload becomes increasingly compute-bound, making the overhead of verifying a large candidate tree more costly. Table 4 evaluates this trade-off across different concurrency levels.

The results show that DARTree delivers its largest gains at low concurrency. As concurrency increases, the benefit of a fixed large tree becomes smaller, while adjusting B and W to the serving load can maintain strong performance. These results highlight the importance of selecting an appropriate tree configuration for the deployment setting.

In summary, the recommended scenarios for DARTree are:

- Local, personal, or small-scale model serving with one or a few concurrent requests, where per-request token throughput is the primary concern.
- Latency-critical or high-priority requests that are processed with small batch sizes.
- Other deployments with underutilized compute resources, where the additional cost of tree-based verification can be absorbed efficiently.

C. Proof of Lemma 1

Proof. Assume that both procedures use the tie-breaking convention in Lemma 1. For any node $u = u_{1:d}$ with parent $p(u)$,

$$s_\beta(u) - s_\beta(p(u)) = \log \tilde{q}_d(u_d | p(u)) + \beta \leq 0,$$

since $\tilde{q}_d(u_d | p(u)) \leq 1$ and $\beta \leq 0$. Scores are therefore non-increasing along every root-to-leaf path and, with ancestors preferred under ties, the global top- B set is prefix-closed. Moreover, any unexposed node has a frontier ancestor in the

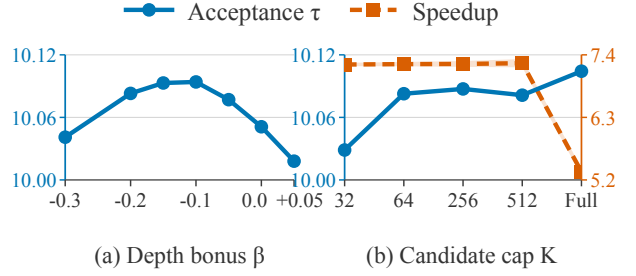


Figure 6: Effect of (a) the depth bonus β and (b) the candidate-vocabulary cap K on Qwen3-4B, averaged equally over GSM8K, HumanEval, and MT-Bench. “Full” denotes the full vocabulary for AR-head correction scoring.

heap with at least the same score. Thus each heap pop returns the next node in the global ordering, and the first B popped nodes are exactly the global top- B nodes.

D. Additional Experiments

Ablations on Depth Bonus and Candidate Sizes. We further examine the depth bonus β and candidate-vocabulary cap K on Qwen3-4B at $T = 0$. As shown in Figure 6, the highest average acceptance length across the three datasets is achieved with negative depth bonuses between -0.2 and -0.1 , while performance remains stable for nearby values. Varying K from 32 to 512 has little effect on acceptance or speedup. However, using the full vocabulary for tree expansion substantially reduces speedup due to the increased latency of the AR head. These results support $\beta = -0.2$ and $K = 64$ as efficient default settings.

Distribution of Verification-Tree Shapes. Figure 7 compares how DDTree and DARTree allocate the verification budget across depths. Across all three benchmarks, DARTree forms narrower trees and allocates more candidates to deeper levels, whereas DDTree more often concentrates its budget in a few wide, shallow levels.

Visualization of Example Trees. Figures 8–11 visualize verification trees generated with Qwen3-4B at $T = 0$ for the prompts “Can you tell me what is the largest animal in the world?” and “What is photosynthesis? Answer in exactly two short sentences.” DDTree contains many repetitive or semantically incompatible parent–child transitions, such as “It can about up” and “plants plants sunlight,” whereas DARTree forms more coherent and well-structured branches. DARTree also yields longer accepted paths in both examples.

E. Implementation Details

We provide additional implementation details of DARTree. Following prior work, the experiments in the main paper use the HuggingFace Transformers backend. For the concurrency evaluation in Table 4, AR, DFlash, and Domino use their official SGLang implementations, while we implement DDTree and DARTree within the same SGLang framework for a fair comparison. We use Triton to accelerate several

Method	$C = 2$	$C = 4$	$C = 8$	$C = 16$	$C = 32$
GSM8K					
AR	159.5 / 1.00×	311.9 / 1.00×	582.0 / 1.00×	1046.8 / 1.00×	1645.8 / 1.00×
EAGLE-3 ($B = 64$)	332.6 / 2.08×	462.9 / 1.48×	535.4 / 0.92×	576.4 / 0.55×	593.9 / 0.36×
DFlash	653.7 / 4.10×	1199.4 / 3.85×	1886.3 / 3.24×	2497.5 / 2.39×	2761.7 / 1.68×
Domino	923.4 / 5.79×	1631.5 / 5.23×	2292.2 / 3.94×	2801.3 / 2.68×	3387.8 / 2.06×
DDTree ($B = 64$)	709.7 / 4.45×	859.5 / 2.76×	1001.3 / 1.72×	1173.7 / 1.12×	1264.8 / 0.77×
DDTree (adaptive B)	709.7 / 4.45×	1314.5 / 4.21×	2197.3 / 3.78×	2614.3 / 2.50×	2873.9 / 1.75×
DARTree ($B = 64, W = 12$)	987.5 / 6.19×	1212.4 / 3.89×	1379.9 / 2.37×	1638.5 / 1.57×	1749.2 / 1.06×
DARTree (adaptive B/W)	987.5 / 6.19×	1766.2 / 5.66×	2806.6 / 4.82×	3395.0 / 3.24×	3701.5 / 2.25×
HumanEval					
AR	159.1 / 1.00×	309.6 / 1.00×	571.3 / 1.00×	977.1 / 1.00×	1301.6 / 1.00×
EAGLE-3 ($B = 64$)	304.9 / 1.92×	422.9 / 1.37×	491.2 / 0.86×	516.0 / 0.53×	540.4 / 0.42×
DFlash	672.3 / 4.23×	1228.2 / 3.97×	2045.4 / 3.58×	2417.4 / 2.47×	2605.5 / 2.00×
Domino	677.1 / 4.26×	1229.8 / 3.97×	1977.8 / 3.46×	2278.3 / 2.33×	2471.3 / 1.90×
DDTree ($B = 64$)	729.9 / 4.59×	882.4 / 2.85×	1016.4 / 1.78×	1201.5 / 1.23×	1276.2 / 0.98×
DDTree (adaptive B)	729.9 / 4.59×	1327.1 / 4.29×	2116.2 / 3.70×	2557.1 / 2.62×	2692.6 / 2.07×
DARTree ($B = 64, W = 12$)	773.0 / 4.86×	938.4 / 3.03×	1101.1 / 1.93×	1304.9 / 1.34×	1391.4 / 1.07×
DARTree (adaptive B/W)	773.0 / 4.86×	1358.9 / 4.39×	2192.8 / 3.84×	2624.3 / 2.69×	2806.9 / 2.16×

Table 4: **Concurrent-serving results on Qwen3-4B at $T = 0$.** All methods are evaluated using SGLang in BF16 on a single NVIDIA RTX 6000 Ada Generation. Throughput is measured as aggregate completion-token TPS using end-to-end request wall-clock time, including prompt prefill. Each cell reports TPS / speedup over AR at the same concurrency. For the adaptive DDTree and DARTree rows, B is set to 64, 32, 16, 16, 16 at $C = 2, 4, 8, 16, 32$, respectively; DARTree additionally uses $W = 12, 8, 3, 3, 3$. The same schedule is used for both datasets.

performance-critical operations, including candidate selection and tree-mask construction. As requested in the reproducibility checklist, we additionally report our system and hardware configuration. All experiments were conducted on a server running Ubuntu 22.04.1 LTS with two AMD EPYC 9374F CPUs, with each experiment using a single NVIDIA RTX 6000 Ada Generation GPU.

F. Future Work

As shown in Figure 5 of the main paper, on some benchmarks, a substantial fraction of decoding rounds reach the end of the available draft block. This concentration near the drafting boundary suggests that the current block length may truncate continuations that could otherwise be accepted. A promising direction is therefore to train drafter models that support longer draft blocks and investigate whether DARTree can translate the expanded drafting horizon into higher acceptance and end-to-end speedups.

Another promising direction is to make the verification budget and tree shape adaptive rather than fixed across requests. These configurations could be selected according to runtime conditions, request load, and content-dependent signals such as draft confidence. Such adaptation may provide a better balance between acceptance gains and drafting and verification overhead across different deployment settings.

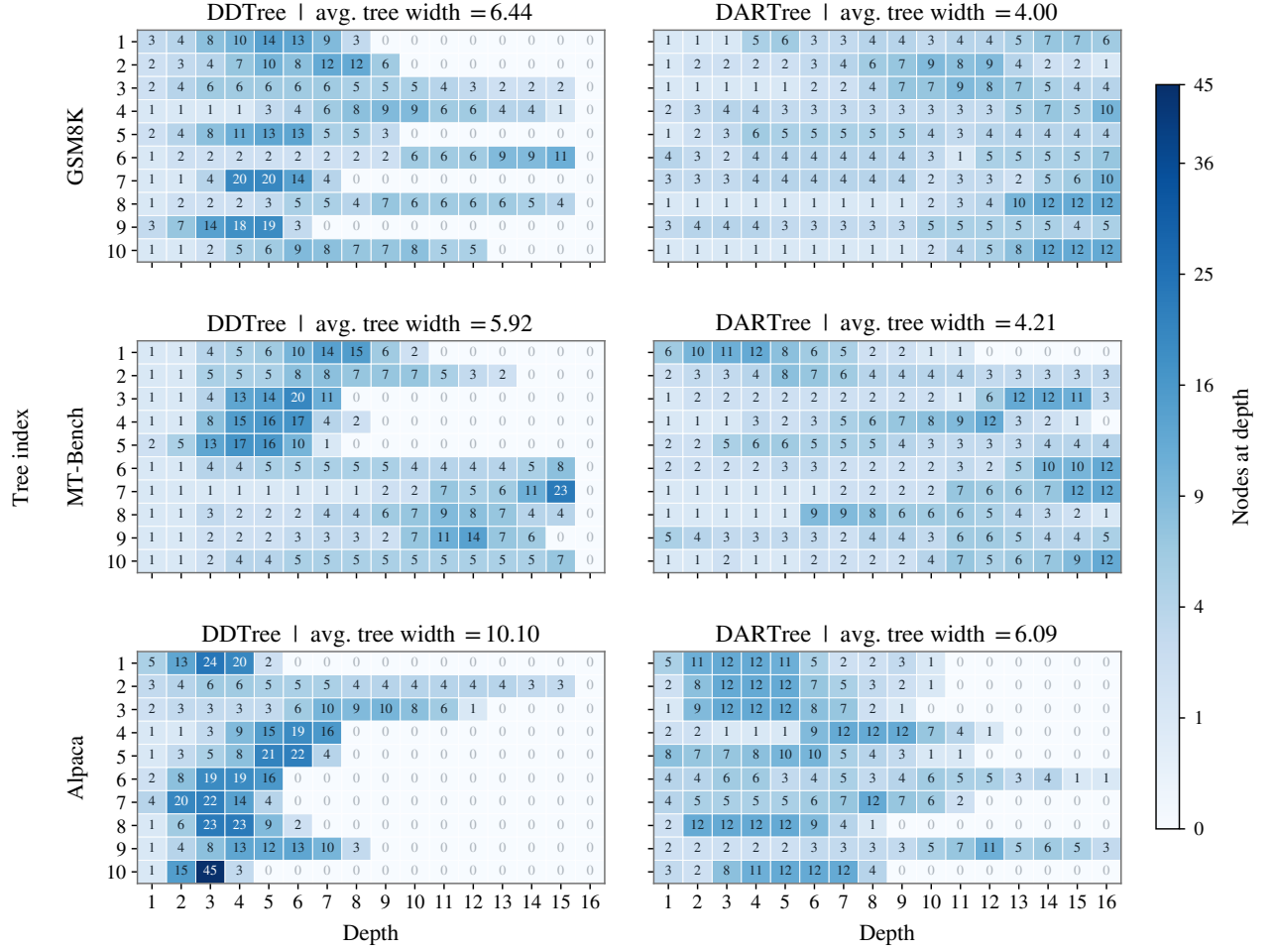


Figure 7: **Node-depth distributions of verification trees.** Each panel contains 10 example trees constructed with budget $B = 64$. Each row denotes an individual tree, and the number in each cell is the number of nodes allocated to that depth.

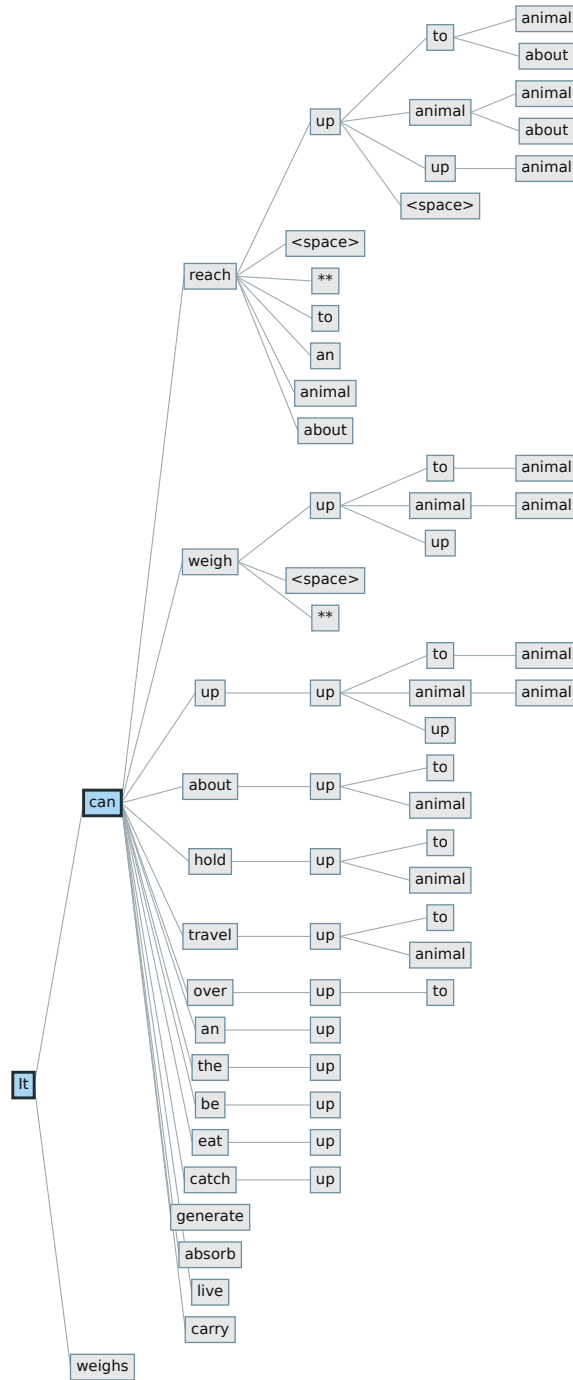


Figure 8: **DDTree verification tree for the largest-animal prompt.** The prompt is “Can you tell me what is the largest animal in the world?” The tree is constructed after the complete prefix “The largest animal in the world is the blue whale (Balaenoptera musculus). Key facts about the blue whale: Size:”. The target model accepts the two-token continuation It can ($\tau = 2$). Blue nodes with thick outlines mark the accepted path; gray nodes with thin outlines mark the remaining candidates. DDTree allocates many nodes to a few depths, producing unusually wide layers.

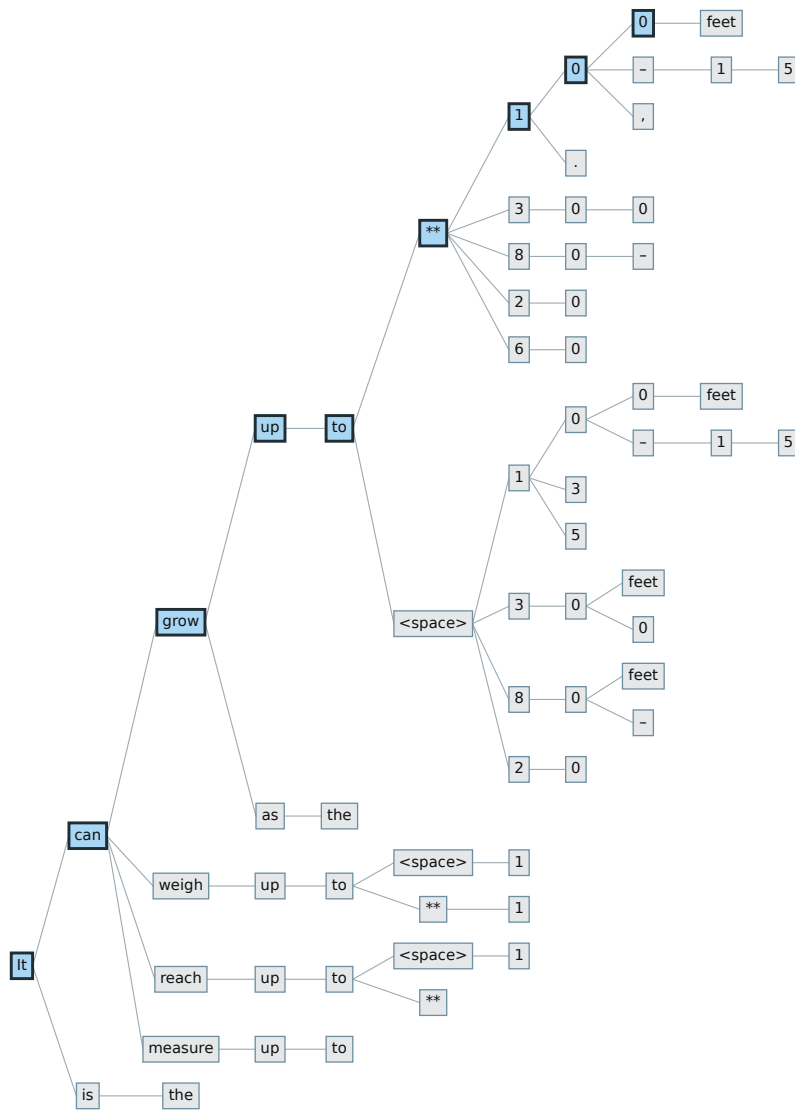


Figure 9: **DARTree verification tree for the largest-animal prompt.** The prompt is “Can you tell me what is the largest animal in the world?” The tree is constructed after the complete prefix “The largest animal in the world is the blue whale (Balaenoptera musculus). Key facts about the blue whale: Size:”. The target model accepts the nine-token continuation It can grow up to **100 ($\tau = 9$). Blue nodes with thick outlines mark the accepted path; gray nodes with thin outlines mark the remaining candidates.

